talenciaglobal
Great talent. Global opportunities

PROCESS PLAYBOOK

SCHEMA-FIRST ENGINEERING WITH AI

# Control the Output

A repeatable process for producing every deliverable with AI: define what you will produce, then make the AI produce exactly that, and prove it.

Discover · Define · Generate · Analyse · Validate · Refine



THE PROBLEM

# Prompting is easy. Knowing what came back is not

A requirements analysis docket was produced from one detailed prompt. It looked impressive. But every section, every column and every row was the LLM's choice. **Nobody had decided what the document should contain, and nobody could explain every field in it.**

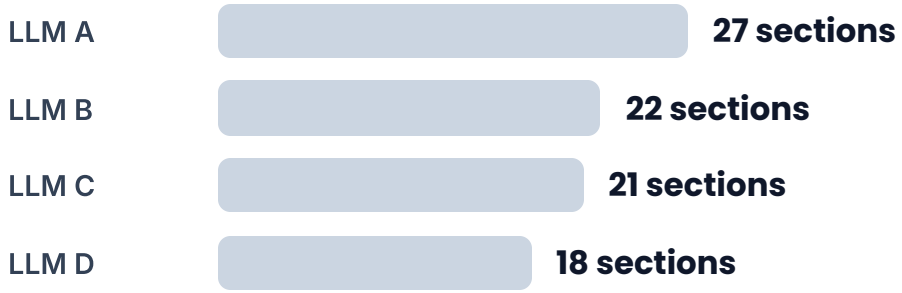
| 27 | 399 | 1,887 | 0 |
|---|---|---|---|
| SECTIONS | PARAMETERS | ROWS OF CONTENT | CHOSEN BY A PERSON |

Measured from the generated workbook (version 0.1).

**If the output is the LLM's choice, the work tests the LLM, not the engineer.** An agent could run the same prompt 36 times. What we need from people is judgement about what to produce, and proof that it is right.

SAME PROMPT, DIFFERENT TOOLS: THE STRUCTURE CHANGES

Run the same prompt in another LLM, or the same LLM twice, and the structure changes. Illustrative:




Ten runs give ten different documents. Nothing built on that can be repeated, reviewed or trusted.

**Uncontrolled**

The LLM decides the shape. The output is accepted as truth. Results differ every run.

**Controlled**

People define the shape. The LLM fills it. Every field is understood, checked and repeatable.

THE PRINCIPLE

# Don't only control the input. Control the output

| | PROMPT-FIRST | SCHEMA-FIRST |
|---|---|---|
| **Starting point** | A prompt with lots of inputs | ✓ The output you need to produce |
| **Who decides the structure** | The LLM, differently each run | ✓ The team, written down as a schema |
| **What the LLM produces first** | The full document | ✓ Only the structure, then content into it |
| **Repeatability** | Different every run | ✓ Same schema every run; content comparable |
| **Review** | Skimmed, if read at all | ✓ Every field read, traced and validated |
| **Cost of a wrong structure** | All content regenerated | ✓ Fixed in the schema before content exists |
| **Role of the person** | Prompt runner | ✓ Engineer: defines, questions and proves |

THE CORE IDEA

## The output schema is the most important artifact of every step.

Know exactly what you will produce before you produce it. Once the schema is frozen, generating content is the easy part.

### One step, several schemas

A step with four deliverables needs four output schemas. Across a full lifecycle, that is dozens of schemas, each defined by a person.

### Structure before content

A schema costs a fraction of the time and tokens of full content. If the structure is wrong, there is no point generating the content.

THE PROCESS AT A GLANCE

# Six stages, one loop, repeated until the output is proven




STAGE 1

## Discover

**What could the output contain?**

LLM proposes · you question

STAGE 2

## Define

**What exactly will we produce?**

You decide

STAGE 3

## Generate

**Fill the schema, nothing else**

LLM produces · you control

STAGE 4

## Analyse

**What did we actually get?**

You read

STAGE 5

## Validate

**Is it right for its purpose?**

You, peers and mentors · scored

STAGE 6

## Refine

**What changes, and where?**

You decide

WHO DOES THE WORK · GREY = LLM, COLOUR = PEOPLE (INDICATIVE)



### Not a straight line

Each pass through the loop improves the schema. Expect two or three passes before an output is stable.

### Every step, every deliverable

Run the loop for each deliverable of each lifecycle step, from the requirements docket to the release plan.

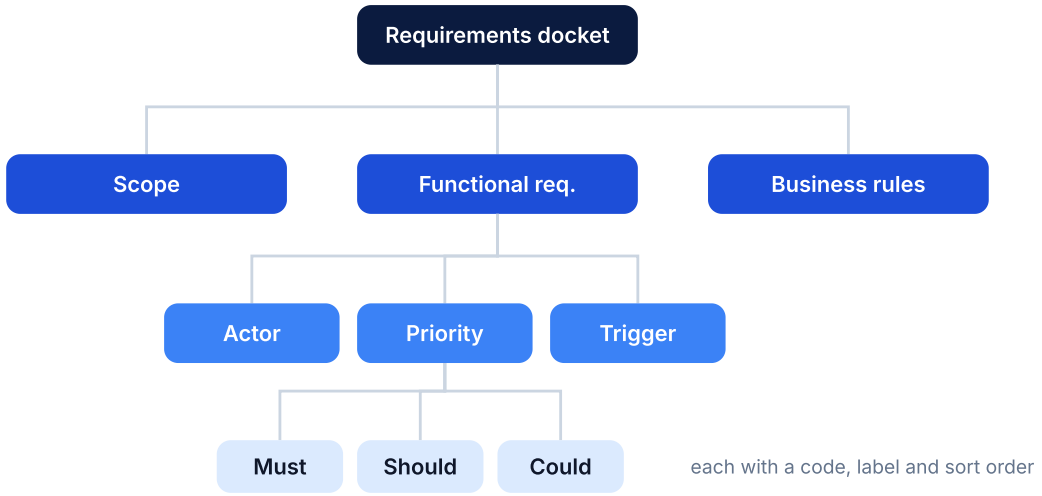
### Closed only when proven

A step stays open until the work that depends on it succeeds. Submitting it is not the same as closing it.

HOW DEEP TO GO

# Every output is organised in levels

A SCHEMA IS A TREE, NOT A FLAT SHEET




Hierarchies are easier to organise, review and connect than one wide sheet. A tree also shows immediately where a section is too thin or too crowded.

FOUR LEVELS: MACRO, MICRO, NANO, PICO

**L1 Macro — Sections**
The major parts of the output.
e.g. Scope · Functional requirements · NFRs · Business rules

**L2 Micro — Parameters**
The fields inside each section.
e.g. Requirement ID · Actor · Trigger · Priority

**L3 Nano — Sub-parameters**
Classifications, allowed values and breakdowns inside a parameter.
e.g. Priority → Must · Should · Could

**L4 Pico — Atomic definition**
Data type, format, validation rule and source of one item.
e.g. Requirement ID → text · pattern FR-000 · unique

DEPTH RULES

- **At least two levels.** One flat list of parameters is never enough.
- **Normally three or four.** Go to five or more only when the content truly needs it.
- **Go deeper when a level is too broad** to reason about: as a guide, more than about 10–15 items under one parent.
- **Stop when each item is atomic:** one meaning, one data type, one rule set.

WORKED EXAMPLE · A REQUIREMENTS ANALYSIS DOCKET, VERSION 0.1

# Measure the output before you trust it

| LEVEL | COUNT | BASIS | QUESTION TO ASK |
|---|---|---|---|
| LEVEL 1 · MACRO | **27** sections | Measured | Are these the right sections? Is any one an overdose? What is missing? |
| LEVEL 2 · MICRO | **399** parameters | Measured · 14.8 per section on average (first rough estimate: 5) | Is 399 parameters in only two levels too broad? Can the team explain each one? |
| LEVEL 3 · NANO | **~150** sub-parameters | Estimate · about 50 parameters with about 3 classifications each | Which parameters have fixed values, ratings or categories that need defining? |
| LEVEL 4 · PICO | **?** to be decided | Not yet discovered | Is a fourth level needed at all? Only Discover and Define can answer that. |

Plus 1,887 rows of generated content that nobody had read line by line.

THE KEY SHIFT

## This document is an input, not an output.

It is where Discover starts: a first view of what a requirements docket could contain. The team still has to decide what it will contain.

### Measure before you judge

Counting sections and parameters takes minutes and immediately shows the scale and shape of what was produced.

### 27 is the LLM's view

It may be right. It may be too many or too few. Until a person has questioned it, nobody knows.

STAGE 1 · DISCOVER

# What could the output contain?

STAGE 1 OF 6

## Discover

Find out what the output could contain before deciding what it will contain.

WHO LEADS

**LLM proposes · you question**

### WHAT YOU DO

- Ask the LLM for the **structure only**: sections, parameters and sub-parameters by level. No content, no data.
- Run it more than once, and in two or three LLMs. Compare what each proposes.
- Study every parameter you do not understand. Ask what it means and why it is needed.
- Re-run with what you learned until a new run adds almost nothing new.

### LLM'S ROLE

Proposes candidate structures and explains unfamiliar terms.

### YOUR ROLE

Questions every section and parameter: what is useless, what is missing, what is grouped wrongly.

### PRODUCE

Candidate schema trees from each run, a glossary of every parameter, and a list of open questions.

### DONE WHEN

You can explain every parameter in your own words, and another run adds almost nothing new.

### PROMPT PATTERN

```
I need to produce a [deliverable] for [context].
Do not generate any content.
Propose a multi-level schema:
· Level 1 sections
· Level 2 parameters per section
· Level 3 sub-parameters or allowed values
· Level 4 where needed
Give a one-line definition and the reason for
each item. Recommend the right depth.
```

**Avoid** Generating the full document at this stage. It costs time and tokens, and the structure is not settled yet.

STAGE 2 · DEFINE

# What exactly will we produce?

STAGE 2 OF 6

## Define

Decide exactly what you will produce, and freeze it as the output schema.

WHO LEADS
You decide

### WHAT YOU DO

- Choose the sections and parameters to keep. Record what you dropped and why.
- Organise them into levels: at least two, normally three or four.
- For every parameter, state its **name, definition, reason, data type, atomic or composite, allowed values, mandatory or optional, and source**.
- **Define the scorecard** the output will be judged on: dimensions, weights, measures and pass thresholds.
- Review both three times, then freeze them together as version 1.

### LLM'S ROLE

When asked, checks the schema for gaps, overlaps and inconsistent naming.

### YOUR ROLE

Makes every decision. Nothing enters the schema unless a person can justify it.

### PRODUCE

Output schema v1, scorecard v1, and a decision log of what was kept, dropped and why.

### DONE WHEN

Every parameter has a definition, a type and a reason, and the scorecard says exactly how the output will be judged.

### REMEMBER

**Frozen is not final. Treat version 1 as about 92% right; generating and validating find the rest.**

STAGE 3 · GENERATE

# Fill the schema, nothing else

STAGE 3 OF 6

## Generate

Produce content that fits the frozen schema exactly, and nothing outside it.

WHO LEADS

**LLM produces · you control**

### WHAT YOU DO

- Give the LLM the frozen schema and the source material. Ask it to fill the schema and nothing else.
- Start small: one section or a few records first, then the rest.
- Run the same schema in two or three LLMs, or have two people run it, so outputs can be compared.
- Tag every output with its source: tool, model, run and date.

### LLM'S ROLE

Fills the schema with content from the source material.

### YOUR ROLE

Controls the scope and inputs, and stops any run that drifts from the schema.

### PRODUCE

Populated output for each run, tagged with its source.

### DONE WHEN

The output follows the schema 100%, and every field holds a value or an explicit UNKNOWN.

### PROMPT PATTERN

```
Using only the attached source and the
attached schema, fill every field.
Do not add, remove or rename any section
or field.
Where the source gives no value, write
UNKNOWN and say why.
Start with section [X] only.
```

**Avoid** Letting the LLM add sections or fields "to be helpful". Any change to the structure goes back to Define.

STAGE 4 · ANALYSE

# What did we actually get?



STAGE 4 OF 6

## Analyse

Open the package. Understand exactly what was generated before anyone uses it.

**WHO LEADS**
You read

**WHAT YOU DO**

- **Open the package.** Read the output field by field, not just the headings.
- Compare runs: where do the LLMs agree, where do they differ, and why?
- Look for vague, duplicated, empty or invented values.
- Measure conformance, fill rate and agreement between runs.

**LLM'S ROLE**

When asked, lists the differences between runs and flags values with no support in the source.

**YOUR ROLE**

Reads, understands and judges every field. This cannot be delegated.

**PRODUCE**

An analysis sheet: notes per parameter, differences between runs, and an issue list.

**DONE WHEN**

You can answer any question about any field: what it means, where it came from, and why it is there.

**ASK YOURSELF**

**What does this value mean? Where did it come from? Would I defend it in a review?**

STAGE 5 · VALIDATE

# Is it right for its purpose?

STAGE 5 OF 6

## Validate

Prove the output is right for its purpose, not just well-formed.

**WHO LEADS**

You, peers and mentors · scored

**WHAT YOU DO**

- **Score the output on its scorecard.** Author first, then a peer independently; discuss any gap of more than one point.
- Trace every item to the source document, or mark it clearly as an assumption.
- Consolidate the best of all runs, and test it in use: does the next step get everything it needs?
- Apply the decision rule: accept, refine or rework. The score decides, not opinion.

**LLM'S ROLE**

Can act as a consolidator across runs, inside the frozen schema only.

**YOUR ROLE**

Author, peer and mentor score each dimension with evidence, and apply the decision rule.

**PRODUCE**

The completed scorecard with evidence per dimension, the decision, and the consolidated version.

**DONE WHEN**

The weighted score is 90 or more, no critical dimension is below 4, and the next step can start from this output.

**REMEMBER**

**You cannot validate what you cannot score. No scorecard, no validation.**

VALIDATE · THE SCORECARD MODEL

# Every output is scored on the same eight core dimensions

| DIMENSION | HOW IT IS MEASURED | WEIGHT | IF LOW, GO BACK TO |
|---|---|---|---|
| **Structure conformance** CRITICAL | Output fields that match the schema ÷ all output fields | **15** | Generate |
| **Completeness** | Fields with a value or an explicit UNKNOWN ÷ all fields; every section covered | **15** | Generate |
| **Definition clarity** | Parameters with definition, type and reason ÷ all parameters | **10** | Define |
| **Accuracy and traceability** CRITICAL | Items traced to the source or marked as assumptions ÷ all items | **20** | Generate |
| **Consistency across runs** | Parameters whose values agree across runs ÷ parameters compared | **10** | Analyse |
| **Structure and depth** | Levels justified; no parent with too many children; every leaf atomic | **10** | Define |
| **Fitness for the next step** | Inputs the next step needs that this output provides ÷ inputs needed | **10** | Define |
| **Author understanding** CRITICAL | Sample questions on fields answered correctly by the author ÷ questions asked | **10** | Discover |
| **Total** | | **100** | |

THE RULE

### You cannot validate what you cannot score.

Every deliverable gets a scorecard. It is defined with the schema in Define and frozen before anything is generated, so the target never moves.

### Core plus specific

These eight core dimensions apply to every deliverable. Add one to three dimensions specific to the deliverable, then re-weight so the total is still 100.

### Critical dimensions

A critical dimension below 4 blocks acceptance, whatever the total. A high average cannot hide an untraceable or misunderstood output.

VALIDATE · SCORING AND THE DECISION RULE

# The score decides whether to accept, refine or rework

## SCORE EACH DIMENSION FROM 0 TO 5

| | | |
|---|---|---|
| 5 | **Excellent** | ≥ 98% |
| 4 | **Good** | 90–97% |
| 3 | **Adequate** | 75–89% |
| 2 | **Partial** | 50–74% |
| 1 | **Poor** | 1–49% |
| 0 | **Absent** | 0% or missing |

For dimensions measured as a percentage, use the ranges shown. For judgement dimensions, score against written evidence, never impression.

## WEIGHTED SCORE OUT OF 100

$$\text{Score} = \Sigma \left( \text{weight} \times \text{dimension score} \div 5 \right)$$

Example: a dimension weighted 20 and scored 4 contributes 20 × 4 ÷ 5 = 16 points.

## DECISION RULE

**Accept**

**Score ≥ 90 and every critical dimension ≥ 4**

Freeze this version. The step stays open until downstream work proves it.

**Refine**

**Score 75–89, or a critical dimension below 4**

Fix the lowest dimensions at the stage the table points to, then re-score.

**Rework**

**Score below 75**

The structure or understanding is wrong. Go back to Discover or Define.

**Two scorers, one evidence base.** The author scores first, then a peer scores independently. Any gap of more than one point is discussed with the evidence; a mentor confirms at checkpoints.

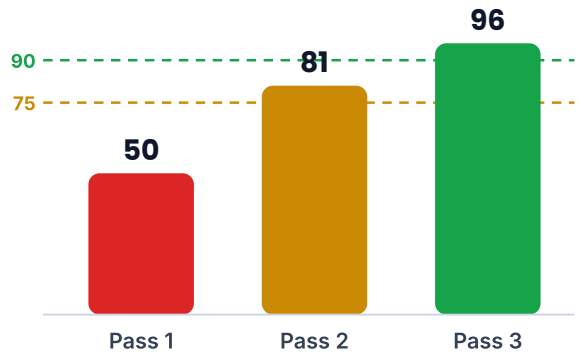
VALIDATE · A SCORECARD ACROSS THREE PASSES

# See where you stand, then decide where to go next

| DIMENSION | WEIGHT | PASS 1 | PASS 2 | PASS 3 |
|---|---|---|---|---|
| Structure conformance * | 15 | 3 | 5 | 5 |
| Completeness | 15 | 3 | 4 | 5 |
| Definition clarity | 10 | 2 | 4 | 5 |
| Accuracy and traceability * | 20 | 2 | 4 | 5 |
| Consistency across runs | 10 | 2 | 3 | 4 |
| Structure and depth | 10 | 3 | 4 | 4 |
| Fitness for the next step | 10 | 3 | 4 | 5 |
| Author understanding * | 10 | 2 | 4 | 5 |
| **Weighted score** | | **50** | **81** | **96** |
| **Decision** | | Rework | Refine | Accept |

* Critical dimension. Scores are illustrative, to show how the model works.

## SCORE TREND ACROSS PASSES




Pass 1 needed rework. Pass 2 was refined on consistency, its weakest dimension. Pass 3 is accepted.

## EXAMPLES OF DELIVERABLE-SPECIFIC DIMENSIONS

**Requirements docket**
Source coverage: every section of the source mapped · Testability of each requirement

**Personas and roles**
Every requirement has an actor · Permission matrix complete

**Feature inventory**
At least three competitors covered · Every feature traced

STAGE 6 · REFINE

# What changes, and where?

STAGE 6 OF 6

## Refine

Decide what changes and where in the loop each change belongs.

**WHO LEADS**
You decide

**WHAT YOU DO**

- Start from the **lowest-scoring dimensions**. Send each finding to **Discover**, **Define** or **Generate**.
- Change the schema or content, and record each change with a reason (v1.1, v1.2, ...).
- Run the loop again and re-score. Repeat until the score passes and changes become small.
- Keep the step open until downstream work proves it: the application works on this data.

**LLM'S ROLE**

Applies agreed changes and regenerates affected parts.

**YOUR ROLE**

Decides what changes, and when the output is good enough to move on.

**PRODUCE**

The next version of the schema and output, a change log, and the score trend across passes.

**DONE WHEN**

Changes between passes are small, and the steps that depend on this output confirm it works.

**REMEMBER**

**Closed means proven in use. A milestone can be reached while the step stays open.**

REFINE · WHERE EACH FINDING GOES

# Every finding goes back to the stage that owns it

| WHAT YOU FOUND | GO BACK TO | EXAMPLE |
|---|---|---|
| **A field nobody can explain** | Discover | "What does this column mean?" with no answer |
| **A missing section or parameter** | Discover | A coverage gap found in peer review |
| **Wrong grouping, or a level that is too broad** | Define | Hundreds of parameters in only two levels |
| **A parameter without a type, values or reason** | Define | "Status" with no list of allowed values |
| **The next step cannot use the output** | Define | The design step needs a field the docket never captured |
| **Output that does not follow the schema** | Generate | Extra columns, renamed fields, merged sections |
| **Vague, invented or untraceable values** | Generate | Tighten the source material and the instructions, then re-run |
| **Runs disagree on a value** | Analyse | Find out why before choosing one |

RULE OF THUMB

**Understanding problem** → Discover.
**Structure problem** → Define.
**Content problem** → Generate.

### Fix it where it starts

Patching content by hand hides a schema problem, and it returns on the next run. Fix the cause at the stage that owns it.

### Version every pass

Each pass gets a version and a line in the change log, so anyone can see what changed and why.

GOING DEEPER, LEVEL BY LEVEL

# Get each level right before opening the next

BREADTH FIRST, THEN DEPTH: THE LOOP RUNS AT EVERY LEVEL

**L1** **Sections** Discover → Define → ... → Refine

Run the loop on Level 1 only. Freeze the section list.

**L2** **Parameters** Discover → Define → ... → Refine

For each section, discover and define its parameters. Freeze.

**L3** **Sub-parameters** Discover → Define → ... → Refine

Add classifications and allowed values where a parameter needs them.

**L4** **Atomic detail** Discover → Define → ... → Refine

Type, format, rules and source, only where needed.

Freeze each level before opening the one below it.

RULES FOR GOING DEEPER

- Don't open the next level until the current one is frozen.
- Don't generate full content until the schema is frozen to the depth you chose.
- If a lower level shows the upper level is wrong, go back up. That is normal, not failure.
- Write down why you stopped at the depth you chose.

HOW LONG IT TAKES

For an experienced engineer, a schema like a requirements docket takes about an hour. Expect longer the first few times. Take the time: it saves days of rework later.

**Precise, not slow**

A team that builds the whole application in two hours spends weeks fixing it. A team that defines every part first finishes sooner.

MEASURABLE RESULTS

# Prove the output with numbers, not impressions

| MEASURE | HOW TO CALCULATE | EXPECTATION |
|---|---|---|
| **Schema size by level** | Count of items at each level | **Known, and every count justified** |
| **Definition coverage** | Parameters with definition, type and reason ÷ all parameters | **100% before generating** |
| **Schema conformance** | Output fields matching the schema ÷ all output fields | **100%** |
| **Fill rate** | Fields with a value or an explicit UNKNOWN ÷ all fields | **100%** |
| **Agreement between runs** | Parameters with matching values across runs ÷ parameters compared | **Tracked; every difference explained** |
| **Traceability** | Items traced to the source or marked as assumptions ÷ all items | **100%** |
| **Review closure** | Review findings closed ÷ findings raised | **100% before moving on** |
| **Change rate** | Parameters added, removed or changed in each pass | **Falling with every pass** |

WHY MEASURE

**An output you cannot measure is an accident, not a result.**

These measures turn "it looks good" into evidence that a reviewer can check.

### They feed the scorecard

Each measure converts to a 0–5 score on its scorecard dimension, so the numbers roll up into one decision.

### Converging is the goal

When the change rate falls pass after pass and the runs agree, the schema is stable enough to rely on.

ACROSS THE DATA-DRIVEN DEVELOPMENT LIFECYCLE

# Every step and every deliverable runs the loop

Discover | Model | Prove | Platform | Insight | Specify | Design | Release × the loop at every step

EXAMPLE · PHASE 1 STEPS AND THE SCHEMAS EACH ONE NEEDS

| STEP | LIFECYCLE STEP | DELIVERABLES (ONE SCHEMA EACH) | LEVEL 1 TO DEFINE, FOR EXAMPLE |
|---|---|---|---|
| 1 | **Requirement Analysis Docket** | Requirements docket | Scope · functional requirements · NFRs · rules · actors · open questions |
| 2 | **Stakeholders, Personas and Roles** | Persona cards · role–permission matrix | Actors · goals · permissions · frequency of use |
| 3 | **Feature Discovery** | Feature inventory · competitor matrix | Features · competitors · coverage · classification |
| 4 | **NFR Discovery** | NFR register | Category · metric · target · test method |
| 5 | **Business Workflow Analysis** | Workflow maps · exception catalogue | Triggers · steps · decisions · exceptions · outputs |

THE ORDER WE BUILD SKILL IN

1 **Engineering** — Master the process and the schemas

2 **Creativity** — Push for standout features

3 **Technology** — Go deep on the stack

WORKING AS A TEAM

# More than one view, compared against one schema

SEVERAL SOURCES OF TRUTH, ONE SCHEMA




Because every run uses the same frozen schema, the outputs can be compared field by field. The consolidator, a person or an agent, works only inside that schema.

### Two people, one item

Two people take the same deliverable independently. Compare the two views and keep the better one, or merge them.

### Peer review

Teams review each other's schemas. Good ideas spread; weak spots are caught before anything is built on them.

### Mentor checkpoints

Mentors review schemas at agreed times and ask each person to explain their choices, field by field.

### A shared schema library

Proven schemas are kept and reused, so the next team starts from a tested base. That is what makes the work repeatable.

ANTI-PATTERNS TO AVOID

# Eight habits that hand control back to the LLM

| ANTI-PATTERN | WHAT IT LOOKS LIKE | STOPPED BY |
|---|---|---|
| ✕ **Courier service** | Passing on output you have not opened or understood | Analyse |
| ✕ **LLM output as truth** | Treating whatever the LLM produced as correct and complete | Analyse · Validate |
| ✕ **Ferrari in two hours** | Building the whole application before its parts are designed | Define, one level at a time |
| ✕ **Written in stone** | Defining once and never revisiting | Refine |
| ✕ **Generate everything first** | Producing full content before the structure is agreed | Discover · Define |
| ✕ **Prompt and paste** | Copying a prompt and submitting whatever comes back | Define |
| ✕ **One flat level** | Hundreds of parameters with no hierarchy | Depth rules |
| ✕ **One run, one view** | No second run or second person, so nothing to compare | Generate · Team |

STAGE CHECKLIST

# One checklist for every deliverable

## STAGE 1 Discover

- [ ] Structure requested, no content
- [ ] Two or more runs or LLMs compared
- [ ] Every parameter understood
- [ ] New runs add almost nothing

## STAGE 2 Define

- [ ] Keep and drop decisions logged
- [ ] Every parameter: definition, type, reason
- [ ] Scorecard defined with weights and thresholds
- [ ] Schema and scorecard frozen as v1

## STAGE 3 Generate

- [ ] Schema and source given, nothing else
- [ ] Small sample first
- [ ] Two or more runs for comparison
- [ ] Every output tagged with source

## STAGE 4 Analyse

- [ ] Every field read
- [ ] Differences between runs explained
- [ ] Vague and invented values logged
- [ ] Conformance and fill rate measured

## STAGE 5 Validate

- [ ] Scored by author and peer, with evidence
- [ ] Score ≥ 90, no critical dimension < 4
- [ ] Traced to source or marked assumption
- [ ] Next step can use it as is

## STAGE 6 Refine

- [ ] Lowest dimensions fixed first
- [ ] Each finding sent to its stage
- [ ] Every change versioned, re-scored
- [ ] Proven by downstream work

**Use this checklist for every deliverable, every pass.**

A box left unticked is the reason to loop back. In reviews, expect to be asked to show the evidence for each tick.

THE MINDSET

# Question every output. Control what you produce.

Whether a person or an LLM produced it, nothing is accepted until it is understood, measured and proven. That is the difference between an engineer and a prompt runner.

**Discover**
Ask for the structure first. Question every part of it.

**Define**
Decide exactly what you will produce and how it will be scored.

**Generate**
Let the LLM fill the schema, and nothing else.

**Analyse**
Open the package. Know every field.

**Validate**
Score it. Prove it against the source, your peers and the next step.

**Refine**
Fix each finding where it started. Frozen is not final.